Data Structures & Algorithms (CST-003)

Complete unit-wise notes - easy English, matched to the UTU syllabus.

Study Materials by Anas - study-materials-by-anas.vercel.app

Unit 1 - Introduction, Analysis & Searching

What this unit covers: what data structures are, how to measure an algorithm (complexity), and the two basic searches.

Key ideas: A **data structure** organises data so operations stay fast; an **ADT** (abstract data type) defines *what* operations exist, not *how* they are coded. **Time complexity** counts how work grows with input size n: O(1) constant, O(log n), O(n), O(n log n), O(n²) - memorise this order. **Big-O** = upper bound (worst case), **Omega** = lower bound, **Theta** = tight bound. **Linear search** checks each element: O(n). **Binary search** needs a sorted array, halves the range each step: O(log n) - learn the mid = (low + high)/2 loop and its dry runs.

Exam pointers: expect "find the complexity of this loop" questions (nested loops on n give O(n²), halving loops give O(log n)) and a binary-search dry run on a given sorted array. Writing linear vs binary search with their conditions is a classic 5-marker.

Unit 2 - Stacks & Queues

What this unit covers: stacks and queues - the two restricted-access lists, with their applications.

Key ideas: A **stack** is LIFO: push and pop happen at the same end (top). Applications: function-call stack, undo, bracket matching, and expression conversion. Learn **infix to postfix** by hand: operands go straight to output; operators go on the stack by precedence (pop higher-or-equal precedence first); '(' pushes, ')' pops until '('. Then **evaluate postfix**: push operands, on an operator pop two, apply, push back. A **queue** is FIFO: insert at rear, delete at front. A **circular queue** wraps the array around to reuse freed slots - know the (index + 1) % size formula and the full/empty conditions. Variants: **deque** (insert/delete at both ends) and **priority queue** (highest priority leaves first).

Exam pointers: converting and evaluating one expression by hand (showing the stack at each step) is almost guaranteed. Also common: write push/pop or enqueue/dequeue as functions, and "why does a simple queue waste space but a circular queue does not".

Unit 3 - Linked Lists

What this unit covers: linked lists - singly, doubly and circular - and how insertion and deletion work without shifting elements.

Key ideas: Each **node** holds data plus a pointer to the next node; the list is reached from a **head** pointer. Advantages over arrays: no fixed size, O(1) insert/delete once you are at the spot. Disadvantages: no random access (must walk from head, O(n)), extra memory per node. Master the pointer steps for: **insert** at beginning / middle / end, and **delete** a node by value - draw the pointers before writing code; the order of pointer updates is what examiners check. A **doubly linked list** adds a prev pointer (walk both directions, more memory); a **circular list** links the last node back to the head.

Exam pointers: "write a function to insert/delete a node" is the standard coding question - practise the singly-linked versions until the pointer order is automatic. Array vs linked list comparison is a favourite theory 5-marker.

Unit 4 - Trees & Graphs

What this unit covers: trees and graphs - hierarchical and networked data, plus the classic walks over them.

Key ideas: A **binary tree** gives each node at most two children. A **BST** (binary search tree) keeps left < root < right, so search/insert take O(height). Traversals: **inorder** of a BST prints values sorted - a very common one-line question. Height-balanced trees (AVL) keep height O(log n). A **graph** is vertices plus edges; store it as an **adjacency matrix** (n x n array, fast edge check, more memory) or **adjacency list** (array of lists, memory-friendly). **BFS** explores level by level using a queue; **DFS** dives deep using a stack/recursion - know which data structure each uses, that question returns every year.

Exam pointers: draw a BST after inserting a given sequence, give its three traversals, and run BFS/DFS on a drawn graph writing the visit order - these three cover most of the unit's marks.

Unit 5 - Sorting & Hashing

What this unit covers: the standard sorting algorithms and hashing - how to order data and how to find it in near-constant time.

Key ideas: Simple sorts: **bubble** (swap neighbours, O(n²)), **selection** (pick the minimum each pass, O(n²)),

insertion (grow a sorted prefix; fast on nearly-sorted data). Fast sorts: **merge sort** (divide, sort halves, merge - always O(n log n), needs extra space) and **quick sort** (partition around a pivot - O(n log n) average, $O(n^2)$ worst on sorted input with bad pivots). **Hashing** maps a key to an index with a hash function (division method: key % tableSize). Collisions are handled by **chaining** (each slot holds a list) or **open addressing** (linear probing: try next slot).

Exam pointers: one full dry run of a sort (usually quick or merge) showing each pass is the repeated numerical; a complexity table of all five sorts (best/average/worst) earns quick marks. For hashing, practise building a table for given keys with linear probing and with chaining.

Made by Anas. Free forever. Watch the linked videos on the site after reading each unit.